TECHNICAL DOCUMENTATION

AI-HRMS: AI-Powered Human Resource Management System

Developer Technical Documentation

> A comprehensive document for the AI-Powered Human Resource Management System, covering module specifications, operational procedures, and system architecture.

Classification: Internal - Development Team
Date: March 2025
Organization: AI-HRMS Product Team

Table of Contents

1. Architecture Overview

1.1 System Architecture

The AI-HRMS platform follows a three-tier architecture: Client Layer (Next.js App Router with React 19 + TypeScript 5), API Layer (Next.js Route Handlers providing RESTful endpoints), and Data Layer (Prisma ORM with SQLite database). External services include z-ai-web-dev-sdk for AI chat completions.

1.2 Data Flow

The application follows a unidirectional data flow pattern: User Interaction triggers Zustand store updates, components invoke REST API endpoints via fetch(), server processing executes Prisma queries, and JSON responses render through Recharts and shadcn/ui components. The /api/ai-chat endpoint proxies user messages to the z-ai-web-dev-sdk for conversational HR assistance with full conversation history support.

1.3 Rendering Strategy

Path	Strategy	Rationale
/ (main page)	Client-side rendering	Dynamic module switching requires client-side interactivity
/api/*	Server-side (Route Handlers)	Direct Prisma access, no client DB exposure
Layout	Server Component	Static shell with fonts, metadata, and toaster

2. Project Structure

The project follows a Next.js App Router structure with organized directories for API routes, business components, UI primitives, hooks, and library utilities.

Directory	Contents	Purpose
prisma/	schema.prisma, seed.ts	Database schema (21 models) and seeder script
src/app/api/	12+ route handlers	RESTful API endpoints for all modules
src/components/hrms/	11 business components	Module-specific UI (Dashboard, Employees, etc.)
src/components/ui/	50+ shadcn/ui primitives	Copy-paste component library with Radix UI
src/hooks/	use-mobile.ts, use-toast.ts	Custom React hooks

Directory	Contents	Purpose
src/lib/	data.ts, db.ts, store.ts, utils.ts	Mock data, Prisma singleton, Zustand store, utilities

3. Technology Stack

3.1 Core Dependencies

Technology	Version	Rationale
Next.js	16.1.1	App Router, React Server Components, Route Handlers
React	19.0.0	Concurrent features, improved hydration, Server Components
TypeScript	5.x	Static type safety across the full stack
Tailwind CSS	4.x	Utility-first CSS with JIT compilation
shadcn/ui	new-york style	Copy-paste component architecture with Radix UI primitives
Prisma	6.11.1	Type-safe ORM with auto-generated client and SQLite support
Zustand	5.0.6	Lightweight client state management with zero boilerplate
Recharts	2.15.4	Declarative charting library built on React components
z-ai-web-dev-sdk	0.0.17	AI chatbot integration with async create() pattern
Lucide React	0.525.0	Consistent icon set with tree-shakeable ESM imports
Zod	4.0.2	Runtime schema validation for API request payloads
next-auth	4.24.11	Authentication framework for future RBAC enforcement
react-hook-form	7.60.0	Performant form handling with minimal re-renders

3.2 Supporting Libraries

Library	Purpose
@tanstack/react-table	Headless table primitives for data grids
@tanstack/react-query	Server state management and caching
@dnd-kit/core + sortable	Drag-and-drop for Kanban boards and pipelines
date-fns	Date manipulation and formatting
framer-motion	Animation primitives for UI transitions

Library	Purpose
sonner	Toast notification system
cmdk	Command palette (Cmd+K) component
next-themes	Dark/light mode theme switching
react-markdown	Markdown rendering for AI chat responses

4. Database Schema

4.1 Entity Relationship Overview

The database consists of 21 Prisma models with Employee as the central entity. Key relationships include: Employee has many Attendance, Leave, Payroll, Expense, Performance, Asset, EmployeeSkill, CourseEnrollment, Document, and AuditLog records. Job has many Candidates. The schema supports audit logging, approval workflows, company policies, shift management, and holiday calendars.

4.2 Key Model Specifications

Employee (Central Entity)

Field	Type	Constraints	Description
id	String	@id @default(cuid())	Primary key
employeeId	String	@unique	Business identifier (e.g., EMP001)
firstName / lastName	String	Required	Employee name
email	String	@unique	Corporate email
department	String?	Optional	Department name (denormalized)
status	String	@default("active")	active, inactive, onboarding, exited
salary	Float?	Optional	Annual salary in INR
createdAt	DateTime	@default(now())	Record creation timestamp
updatedAt	DateTime	@updatedAt	Auto-updated modification timestamp

Other Key Models

Model	Key Fields	Purpose
Attendance	employeeId, date, checkIn, checkOut, status	Daily attendance tracking

Model	Key Fields	Purpose
Leave	employeeId, leaveType, startDate, endDate, status	Leave management
Payroll	employeeId, month, year, basicSalary, hra, netPay, status	Payroll processing
Expense	employeeId, category, amount, date, status	Expense claims
Performance	employeeId, reviewPeriod, rating, attritionRisk	Performance reviews
Job	title, department, location, type, skills, status	Job postings
Candidate	jobId, name, aiFitScore, status	Candidate pipeline
AuditLog	userId, action, module, details, ipAddress	Audit trail
Course	title, category, duration, skills	Learning catalog
ApprovalWorkflow	type, requesterId, approverId, status	Multi-level approvals

5. API Reference

5.1 Common Patterns

All API endpoints follow RESTful conventions and return JSON. All write operations create AuditLog entries. List endpoints support pagination with page and limit query parameters (default page=1, limit=10). Most list endpoints support field-specific query parameters for filtering.

5.2 Endpoint Summary

Endpoint	Methods	Key Features
/api/employees	GET, POST	List (paginated, searchable) and create employees
/api/employees/[id]	GET, PUT, DELETE	Single employee CRUD with nested relations
/api/attendance	GET, POST	Attendance records with date range filtering
/api/leaves	GET, PATCH	Leave management; PATCH for approve/reject
/api/payroll	GET, POST	Payroll processing with auto-calculation
/api/expenses	GET, PATCH	Expense claims with status transition rules
/api/performance	GET	Performance review data with filtering
/api/jobs	GET	Job postings with nested candidate data
/api/candidates	GET	Candidate pipeline with AI fit scores
/api/courses	GET, POST	Dual-mode: catalog or enrollment operations

Endpoint	Methods	Key Features
/api/audit	GET	Audit log with comprehensive filtering
/api/dashboard	GET	Aggregation: 12 parallel Prisma queries
/api/ai-chat	POST	AI chatbot with conversation history

6. Component Architecture

6.1 Module Registry Pattern

The application uses a module registry pattern. page.tsx maps ModuleKey strings to component classes. The active module is determined by useHRMSStore().activeModule, and the corresponding component is rendered dynamically.

6.2 Component Specifications

Component	Module Key	AI-Powered	Primary Data Sources
Dashboard	dashboard	No	/api/dashboard
EmployeeManagement	employees	No	/api/employees, mock data
RBACSecurity	rbac	No	Mock roles/permissions data
TalentAcquisition	talent	Yes	/api/jobs, /api/candidates
TimeAttendance	attendance	No	/api/attendance, /api/leaves
PayrollExpense	payroll	No	/api/payroll, /api/expenses
Performance	performance	Yes	/api/performance
LearningDevelopment	learning	Yes	/api/courses
Analytics	analytics	Yes	/api/dashboard, mock data
SelfService	selfservice	No	/api/ai-chat, employee data

7. Authentication and Authorization

7.1 Authentication

The application integrates next-auth v4 for authentication infrastructure. The current implementation uses a simplified session model with plans for full OAuth2/OIDC integration.

7.2 Role Hierarchy

Level	Role	User Count	Scope
0	Super Admin	1	Full system access, all modules, all permissions
1	HR Admin	3	HR, Payroll, Attendance, Performance (read/write/modify)
2	Payroll Specialist	2	Payroll (read/write/modify), HR (read), Analytics (read)
2	Recruiter	4	HR (read/write/modify), Learning (read), Analytics (read)
2	L&D; Manager	2	Learning (full CRUD), Performance (read/write/modify)
3	Department Manager	8	Team management, leave/expense approval, reviews
4	Employee	140	Self-service read access, learning enrollment

8. AI Integration

8.1 Chat Architecture

The AI chat endpoint (/api/ai-chat/route.ts) implements: request validation, HR-specific system prompt defining the AI persona across 8 domains, conversation history maintenance (user and assistant roles), SDK initialization using the async ZAI.create() factory pattern, response extraction with fallback error handling, and comprehensive error catching with descriptive 500 status messages.

8.2 AI-Powered Features

Feature	Module	Description
AI Chat Assistant	Self-Service	Conversational HR helpdesk via z-ai-web-dev-sdk
AI Fit Score	Talent Acquisition	0-100 candidate-job compatibility scoring
AI Course Recommendations	Learning	Skill-gap-based course suggestions
AI Attrition Analysis	Performance	Department-wise attrition risk prediction
AI Interview Questions	Talent Acquisition	Auto-generated questions tailored to job and candidate
AI Resume Matching	Talent Acquisition	Multi-dimensional match analysis with progress bars
AI Compliance Bot	Talent Acquisition	Real-time onboarding compliance status checking

9. State Management

Client-side state is managed via Zustand (v5.0.6). The global store (src/lib/store.ts) manages: activeModule (determines which module component is rendered), sidebarOpen (mobile sidebar drawer state), and searchQuery (sidebar module search filter). Server state is managed through direct fetch() calls to API routes, with components handling loading, error, and success states locally.

10. Styling System

The application uses Tailwind CSS v4 with JIT compilation for utility-first styling. Global CSS variables are defined in globals.css for theming. The shadcn/ui component library provides pre-built, accessible components using Radix UI primitives. The theme supports dark/light mode switching via next-themes. The primary accent color is Emerald (#10b981), used consistently across AI badges, active states, and interactive elements.

11. Testing Strategy

The testing strategy encompasses: Unit Testing (Jest + React Testing Library for component logic and utility functions), Integration Testing (API route handler testing with Prisma test database), End-to-End Testing (Playwright for critical user flows including login, CRUD operations, and AI chat), and Type Checking (TypeScript strict mode with no implicit any for compile-time safety).

12. Deployment Guide

The application is built on Next.js 16 with built-in optimization for deployment. Key deployment considerations: Node.js 18+ runtime requirement, SQLite database file persistence in the db/ directory, environment variable configuration for API keys and database URLs, and Prisma migration execution on deployment. The application supports standard Next.js deployment targets including Vercel, Docker containers, and Node.js server environments.

13. CI/CD Pipeline

The CI/CD pipeline includes: Lint (ESLint with flat config for code quality), Type Check (TypeScript compiler for type safety), Test (Jest for unit and integration tests), Build (Next.js production build), and Deploy (automated deployment to the target environment). Pipeline configuration follows GitHub Actions or equivalent CI/CD platform standards.

14. Environment Variables

Variable	Purpose	Required
DATABASE_URL	Prisma connection string for SQLite	Yes
NEXTAUTH_SECRET	Secret key for next-auth session encryption	Yes
NEXTAUTH_URL	Base URL for next-auth callbacks	Yes
ZAI_API_KEY	API key for z-ai-web-dev-sdk	Yes (for AI features)
NEXT_PUBLIC_APP_URL	Public application URL	No

15. Contributing Guidelines

Contributions follow a structured workflow: Fork and clone the repository, create a feature branch from the develop branch, implement changes with appropriate test coverage, ensure all lint and type checks pass, submit a pull request with a descriptive title and summary, and obtain at least one code review approval before merging. Commit messages should follow Conventional Commits format (feat:, fix:, docs:, etc.).

16. Code Style Guide

Area	Standard
Language	TypeScript strict mode (no implicit any)
Framework	Next.js App Router conventions (Server/Client Components)
Components	Functional components with hooks; named exports
Styling	Tailwind CSS utility classes; shadcn/ui for complex components
State	Zustand for global state; local state for component-specific data
API	RESTful Route Handlers with Zod validation
Naming	camelCase for variables/functions, PascalCase for components/types
Imports	Absolute paths using @/ alias for src/ directory